MAHDEPLOY | USER GUIDE | V1.3.1

استقراربان

MahDeploy — FastAPI + React Deployer for Windows Server

راهنمای جامع کاربر

استقرار استاندارد و خودکار اپلیکیشن‌های FastAPI + React روی ویندوز سرور؛ با پیش‌بررسی کامل دامنه، پورت و گواهی SSL، اسکریپت‌های استاندارد PowerShell، وب‌سرور Nginx و سرویس ویندوز. در نسخه ۱.۳.۰ سه قابلیت بزرگ اضافه شده است: گزارش دقیق خطا با علت و راه‌حل پیشنهادی، بررسی سلامت چندسطحی سامانه‌های مستقرشده و به‌روزرسانی با نسخه‌بندی + بازگشت سریع به نسخهٔ سالم؛ و نسخهٔ ۱.۳.۱ تشخیص هوشمند تداخل پورت بک‌اند (شناسایی پردازش اشغال‌کننده با دستور توقف دقیق) و رفع چند خطای اسکریپتی را اضافه می‌کند. فصل ۵ این سند، مرجع کامل اسکریپت‌های PowerShell است.

نسخه سند: **۱.۳.۱**

تاریخ انتشار: **آبان ۱۴۰۵**

مخاطب: **مدیران سرور و تیم‌های فنی**

سیستم‌عامل: **Windows Server ۲۰۱۶+**

سند رسمی محصول — همراه پکیج پرتابل

تهیه شده توسط شرکت ماه افزارهوشمند رسا

github.com/hjalilvand2004-coder/MahDeploy

MahAI.ir

TABLE OF CONTENTS

فهرست مطالب

هشت فصل، از آشنایی با سامانه تا مرجع کامل اسکریپت‌ها و عیب‌یابی — هر بخش با همان زبانی که در خود پنل می‌بینید.

۱ **آشنایی با استقراربان** — چرا این سامانه؟ معماری، امکانات کلیدی و جریان کار ۳

۲ **اجرا و راه‌اندازی** — اجرای پکیج پرتابل، ثبت سرویس ویندوز و نخستین ورود ۴

۳ **داشبورد سلامت سرور** — متریک‌های زنده، وضعیت Nginx، نقشه پورت‌ها و رویدادها ۵

٤ **ویزارد استقرار — گام‌به‌گام** — پنج مرحله استقرار، بررسی‌های پیش‌نیاز و ترمینال زنده ۶

۵ **الگوهای اسکریپت‌های PowerShell** — ترتیب اجرا، نام مرحلهٔ هر اسکریپت و دقیقاً چه کاری روی سرور انجام می‌دهد ۸

٦ **مدیریت سامانه‌های مستقرشده** — سلامت چندسطحی، به‌روزرسانی نسخه‌دار و بازگشت به نسخهٔ سالم، کنترل سرویس‌ها و حذف کامل ۱۸

۷ **امنیت و نگهداری** — رمز عبور، فایروال، چرخش لاگ‌ها، پشتیبان‌گیری و دسترسی Administrator ۲۰

۸ **عیب‌یابی و پرسش‌های رایج** — جدول مشکلات متداول با علت و راه‌حل + خواندن گزارش خطای پنل ۲۱

CHAPTER 01

آشنایی با استقراربان

چرا استقرار دستی روی ویندوز سرور دردسر دارد و این سامانه چه چیزی را استاندارد می‌کند.

استقرار دستی یک اپلیکیشن **FastAPI + React** روی ویندوز سرور یعنی ده‌ها کار ریز و حساس: ساخت ساختار پوشه، ساخت محیط مجازی پایتون، نصب وابستگی‌ها، بیلد فرانت‌اند، تنظیم Nginx و SSL، باز کردن فایروال و ثبت سرویس ویندوز — که هر کدام جای خودش را دارد و یک اشتباه کوچک، سامانه را از دسترس خارج می‌کند. **استقراربان (MahDeploy)** این فرآیند را به یک ویزارد ۵ مرحله‌ای تبدیل می‌کند: شما مشخصات را وارد می‌کنید، سامانه همه‌چیز را پیش‌بررسی می‌کند و تا همه بررسی‌ها سبز نشوند اجازه نصب نمی‌دهد؛ سپس با اسکریپت‌های استاندارد PowerShell عملیات را اجرا و گزارش زنده نشان می‌دهد.

پنل کاملاً **پرتابل** است: نیازی به نصب پایتون، Node.js، PyCharm یا دیتابیس روی سیستمِ اجراکننده پنل نیست و همه ران‌تایم‌ها داخل پکیج قرار دارند. رابط کاربری فارسی و راست‌به‌چپ است و هر صفحه، دکمه «راهنما»ی خودش را دارد.

معماری ترافیک پس از استقرار



پیش‌بررسی پیش از نصب

DNS دامنه، آزاد بودن پورت، اعتبار و تطابق گواهی SSL و وجود مسیر کد — همه به‌صورت زنده و قبل از هر تغییری روی سرور کنترل می‌شوند.

اسکریپت‌های استاندارد

۶ اسکریپت PowerShell استاندارد برای هر استقرار تولید می‌شود؛ قابل مشاهده، کپی و دانلود — شفافیت کامل درباره «چه چیزی روی سرورم اجرا می‌شود».

Nginx و SSL خودکار

کانفیگ HTTPS روی پورت ۴۴۳ با گواهی شما، پروکسی /api به بک‌اند و سرو فایل‌های React به‌صورت خودکار ساخته و ریلود می‌شود.

سرویس ویندوز با WinSW

بک‌اند FastAPI به‌عنوان سرویس ویندوز ثبت می‌شود؛ با بوت سرور بالا می‌آید و در صورت کرش، خودکار دوباره اجرا می‌گردد.

پایش زنده سرور

مصرف CPU، رم و دیسک، وضعیت Nginx و نقشه کامل پورت‌های اشغال‌شده، هر ۴ ثانیه به‌روزرسانی می‌شود.

مدیریت چرخه حیات

شروع، توقف، ری‌استارت و حذف کامل هر سامانه مستقرشده — همراه با آرشیو لاگ‌ها و پاک‌سازی کانفیگ‌ها.

گزارش خطای قابل ردیابی

اگر هر اسکریپتی در اجرا خطا بدهد، پنل متن خطا، علت احتمالی و راه‌حل‌های پیشنهادی فارسی را کنار همان مرحله نشان می‌دهد و در تاریخچه رویدادها ثبت می‌کند.

به‌روزرسانی بدون دلهره

نسخه جدید کنار نسخه فعال آماده و تست می‌شود و نسخهٔ فعال تا سوییچ نهایی دست‌نخورده می‌ماند؛ با تاریخچه نسخه‌ها، بازگشت سریع به نسخهٔ سالم همیشه ممکن است.

CHAPTER 02

اجرا و راه‌اندازی

از بازکردن ZIP تا نخستین ورود موفق — در کمتر از پنج دقیقه.

پکیج پرتابل هیچ پیش‌نیازی ندارد؛ فقط به **ویندوز ۱۰/۱۱ یا ویندوز سرور ۲۰۱۶ به بالا (٦٤ بیتی)** نیاز دارید. کل پوشه را در مسیری بدون فاصله و بدون حرف فارسی باز کنید — پیشنهاد ما `C:\MahDeploy` است.

۱ اجرای پنل

روی `Start-MahDeploy.bat` دوبار کلیک کنید. پنل پورت ۳۰۰۰ را چک می‌کند، دیتابیس آماده می‌شود و مرورگر به‌صورت خودکار روی `http://localhost:3000` باز می‌شود. برای پورت دلخواه: `Start-MahDeploy.bat 8080`

۲ نخستین ورود

با رمز پیش‌فرض `admin1234` وارد شوید. صفحه ورود، دکمه «راهنمای ورود و شروع سریع» دارد که همین سه گام را داخل پنل هم نشان می‌دهد.

۳ تغییر رمز عبور

از دکمه چرخ‌دنده در نوار بالا، رمز جدید با حداقل ۸ کاراکتر ثبت کنید. این کار را قبل از هر استقرار واقعی انجام دهید.

٤ (اختیاری) ثبت به‌عنوان سرویس ویندوز

در PowerShell با دسترسی Administrator داخل پوشه پکیج: `powershell -ExecutionPolicy Bypass -File .\service-install.ps1` — پنل با نام سرویس «MahDeploy Panel» ثبت می‌شود، با بوت سرور بالا می‌آید و پس از کرش خودکار ری‌استارت می‌شود. حذف: `service-uninstall.ps1`

۵ (اختیاری) دسترسی از شبکه

برای باز کردن پنل از سایر سیستم‌ها، پورت را در فایروال باز کنید:

```
New-NetFirewallRule -DisplayName "MahDeploy Panel" -Direction Inbound -Protocol TCP -LocalPort 3000 -Action Allow
```

ساختار پکیج: پوشه `app` برنامه پنل، `runtime` ران‌تایم Node.js، `data\mahdeploy.db` دیتابیس پنل و `logs` لاگ‌های اجراست. پشتیبان‌گیری کامل = کپی کل پوشه.

دسترسی Administrator: برای اجرای عملیات واقعی استقرار (PowerShell، سرویس ویندوز، Nginx) پنل را در نشستی با دسترسی ادمین اجرا کنید؛ در غیر این صورت پنل فقط حالت نمایشی دارد.

CHAPTER 03

۰۳ داشبورد سلامت سرور

نگاه اول: سرور حالا در چه وضعیتی است و چه چیزی روی آن در حال اجراست.

داشبورد نخستین صفحه پس از ورود است و داده‌هایش هر ۴ ثانیه تازه می‌شود. چهار کارت بالای صفحه وضعیت لحظه‌ای را نشان می‌دهند و ردیف‌های پایین، جزئیات محیط اجرا و رویدادهای اخیر را.

ویجت	چه چیزی نشان می‌دهد	چگونه بخوانیم
پردازنده	درصد مصرف فعلی CPU با نمودار روند اخیر و تعداد هسته‌ها	عدد بزرگ درصد مصرف است؛ نمودار کوچک، چند دقیقه اخیر را خلاصه می‌کند
حافظه RAM	درصد مصرف رم و مقدار مصرف‌شده از کل (گیگابایت)	اگر بالای ۸۵٪ پایدار بماند، برای استقرار جدید جا باز کنید
دیسک	درصد اشغال دیسک اصلی و فضای آزاد باقی‌مانده	نوار رنگی زیر عدد، نسبت اشغال را بصری نشان می‌دهد
Nginx	نصب و فعال‌بودن وب‌سرور، نتیجه تست کانفیگ، تعداد سایت‌های فعال	حالت قرمز یعنی Nginx نصب نیست یا اجرا نمی‌شود — پیش‌نیاز استقرار
محیط اجرا	نسخه Python، Node.js، Nginx و آپ‌تایم سرور	موارد سبز آماده استقرارند؛ «نصب نیست» یعنی باید پیش از استقرار نصب شود
سامانه‌های مستقر	خلاصه تعداد فعال/ناموفق/کل به‌همراه فهرست کوتاه	دکمه «مدیریت سامانه‌ها» به تب مربوطه می‌رود
نقشه پورت‌ها	همه پورت‌های در حال شنیدن با نام فرآیند و PID	پورت‌های مربوط به استقرارها با برچسب فیروزه‌ای مشخص شده‌اند
جریان رویدادها	آخرین عملیات‌ها: ورود، تولید پلن، اجرای اسکریپت‌ها، خطاها	رنگ نقطه کنار هر رویداد، سطح آن (موفق/خطا/اطلاع) را نشان می‌دهد

دکمه «راهنمای داشبورد» در بالای همین صفحه، معرفی کوتاه‌تری از همین ویجت‌ها را داخل پنل نشان می‌دهد.

CHAPTER 04

ویزارد استقرار — گام‌به‌گام

قلب سامانه: از هویت سامانه تا ترمینال زنده، در پنج مرحله کنترل‌شده.

در تب «استقرار جدید» یک نوار پیشرو (Stepper) می‌بینید که جایگاه شما را نشان می‌دهد و زیر آن دکمه «**راهنمای مرحله X**» قرار دارد که همان مرحله را داخل پنل توضیح می‌دهد. تا مرحله قبل کامل نشود، مرحله بعد باز نمی‌شود — این محدودیت عمدی است تا هیچ استقراری با اطلاعات ناقص شروع نشود.

۱ هویت سامانه

«نام نمایشی» فارسی برای پنل است؛ «شناسه سامانه» اسلاگ انگلیسی (مثل `docs-manager`) که خودکار ساخته می‌شود و در نام سرویس ویندوز، مسیر پوشه‌ها و فایل کانفیگ Nginx به کار می‌رود — به همین دلیل فقط حروف کوچک انگلیسی، عدد و خط تیره می‌پذیرد. منبع کد را هم اینجا انتخاب می‌کنید: «مسیر پوشه روی سرور» یا «آپلود ZIP» تا ۳۰۰ مگابایت. ساختار استاندارد کد: پوشه `backend` برای FastAPI و `frontend` برای React.

۲ دامنه و پورت

دامنه‌ای که از قبل خریده‌اید و رکورد A آن به IP همین سرور اشاره می‌کند. پورت بک‌اند را می‌توانید دستی بدهید یا با دکمه پیشنهاد، نخستین پورت آزاد (از ۸۰۰۱ به بعد) خودکار وارد شود. دکمه‌های «بررسی» بلافاصله رزولوشن DNS و آزاد بودن پورت را می‌سنجند و نتیجه زیر فیلد نمایش داده می‌شود. پیش‌نمایش معماری پایین صفحه، مسیر ترافیک را قبل از نصب نشان می‌دهد.

۳ مسیرها و گواهی SSL

مسیر پایه استقرار (پیش‌فرض `C:\deployments`)، مسیر نصب Nginx (پیش‌فرض `C:\nginx`) و مسیر دو فایل گواهی (`.crt/.pem` و `.key`) دامنه. با دکمه «اعتبارسنجی گواهی SSL» وجود فایل‌ها و تطابق دامنهٔ گواهی کنترل می‌شود. زیر هر سامانه این ساختار ساخته می‌شود:

standard deployment layout — C:\deployments\<name>

```
C:\deployments\docs-manager\
├── backend\        ← FastAPI + .venv (virtual environment)
├── frontend\       ← React source
├── www\            ← production build (served by Nginx)
├── service\        ← WinSW service wrapper
├── logs\           ← app & service logs
└── scripts\        ← generated PowerShell scripts
```

۴ بررسی نهایی پیش از استقرار

همه پیش‌نیازها یک‌جا و زنده کنترل می‌شوند. معنی رنگ‌ها:

سبز — موفق
بررسی پاس شد؛ می‌توانید ادامه دهید.

زرد — هشدار
مانع جدی نیست (مثل DNS تازه ثبت‌شده که هنوز منتشر نشده) ولی بدانید چه می‌کنید.

قرمز — قطع‌کننده
استقرار ممکن نیست؛ مورد را اصلاح و دوباره «بررسی مجدد» بگیرید.

وقتی همه موارد سبز شد، دکمه «**تولید پلن PowerShell استقرار**» فعال می‌شود. پلن یعنی ۶ اسکریپت استاندارد که به‌ترتیب اجرا خواهند شد — و پیش از اجرا، محتوای کامل هرکدام قابل مشاهده، کپی و دانلود است (شرح دقیق و کامل هر شش اسکریپت، به‌همراه اسکریپت‌های مدیریتی و به‌روزرسانی، در فصل ۵ آمده است):

اسکریپت	کار دقیق آن روی سرور
`01-preflight.ps1`	چک نهایی لحظه اجرا: پورت، دیسک، دسترسی نوشتن، وجود Python و Nginx
`02-prepare.ps1`	ساخت ساختار پوشه استاندارد و کپی/استخراج کد منبع به مسیر نهایی
`03-backend.ps1`	ساخت محیط مجازی، نصب وابستگی‌ها و ثبت سرویس ویندوز با WinSW
`04-frontend.ps1`	نصب پکیج‌ها، بیلد React و انتقال خروجی به پوشه www
`05-nginx.ps1`	نوشتن کانفیگ HTTPS، افزودن قاعده فایروال، تست و ریلود Nginx
`06-verify.ps1`	تست نهایی HTTP و وضعیت سرویس؛ تأیید سلامت کل زنجیره

۵ اجرا و گزارش زنده

چک‌باکس تأیید را بزنید و «اجرای استقرار» را کلیک کنید. اسکریپت‌ها با دسترسی Administrator به‌ترتیب اجرا می‌شوند و هر خط خروجی، همان لحظه در ترمینال رنگ‌بندی‌شده دیده می‌شود (سبز موفق، زرد هشدار، قرمز خطا). با کلیک روی هر مرحله در ستون کناری، کد کامل همان اسکریپت باز می‌شود. در پایان موفق، لینک مستقیم `https://دامنه` ، نام سرویس و مسیر کانفیگ Nginx نمایش داده می‌شود و کارت سامانه در تب «سامانه‌ها» ثبت می‌گردد.

اگر مرحله‌ای خطا بدهد (جدید در ۱.۳.۰): پنل اجرا همان‌جا متوقف می‌شود و به‌جای یک خروجی خام، کارت «تحلیل خطا» باز می‌شود که سه بخش دارد: **۱) متن خطا** — آخرین خطوط قرمز [FAIL] همان مرحله، عیناً از خروجی اسکریپت؛ **۲) علت احتمالی** — جملهٔ فارسی و قابل فهم (مثلاً «فایل ZIP خراب یا ناقص آپلود شده است» یا «کلید خصوصی با گواهی SSL مطابقت ندارد»)؛ **۳) راه‌حل‌های پیشنهادی** — گام‌به‌گام، متناسب با همان خطا. دکمه «تلاش دوباره از مرحلهٔ ناموفق» هم وجود دارد که مراحل موفق قبلی را تکرار نمی‌کند. متن کامل خطا + علت، در رویدادهای همان سامانه هم ثبت می‌شود تا بعداً قابل ردیابی باشد.

دو نکتهٔ امنیتی دربارهٔ خروجی اسکریپت‌ها: اول اینکه **اطلاعات حساس** (پسورد، توکن، کلید API و مقادیر مشابه که در خروجی اسکریپت‌ها ظاهر شوند) پیش از نمایش و پیش از ثبت در دیتابیس پنل، به‌صورت خودکار با *** ماسک می‌شوند؛ دوم اینکه گزارش کامل و بدون فیلتر هر مرحله در فایل ترنسکریپت همان مرحله روی سرور (`C:\ProgramData\DeployGuard\logs`) ذخیره می‌شود — برای پیوست‌کردن به گزارش خطا، مسیر فایل در همان خروجی نمایش داده شده است.

فلسفه استقراربان: تا همه بررسی‌ها سبز نشوند هیچ خطی روی سرور اجرا نمی‌شود؛ و پیش از اجرا هم همه‌چیز با چشم خودتان قابل بازبینی است.

CHAPTER 05

الگوهای اسکریپت‌های PowerShell — مرجع کامل

۰۵

هر پلن استقراربان از کدام اسکریپت‌ها ساخته شده، به چه ترتیبی اجرا می‌شود و هر کدام دقیقاً چه کاری روی سرور انجام می‌دهد.

پنل هیچ دستور پنهانی روی سرور اجرا نمی‌کند؛ برای هر سامانه و از روی همان مقادیری که در ویزارد وارد کرده‌اید (نام، دامنه، پورت، مسیرها، گواهی SSL و ...) اسکریپت‌های استاندارد PowerShell تولید می‌شوند. این اسکریپت‌ها پیش از اجرا در گام «تولید پلن» قابل مشاهده، کپی و دانلود هستند، روی سرور در پوشهٔ `scripts\` همان سامانه ذخیره می‌شوند و به‌ترتیبِ همین فصل با دسترسی Administrator اجرا می‌گردند. متن کامل هر اسکریپت در هر لحظه از تب «اسکریپت‌های PowerShell» در پنجرهٔ جزئیات سامانه هم در دسترس است.

همهٔ اسکریپت‌های استقرار، به‌روزرسانی، سلامت و نگهداری یک **اسکلت مشترک** دارند که رفتار یکنواخت گزارش خطا و لاگ‌گیری را تضمین می‌کند: بلوک تنظیمات `$cfg`، کنترل دسترسی Administrator، گزارش خطای تک‌خطی به‌جای stack trace خام، خروجی UTF-۸ و ترنسکریپت کامل مرحله:

common skeleton — identical in every generated script

```
param([switch]$Silent)
$ErrorActionPreference = 'Stop'
trap {                    # clean one-line error report, not a raw stack trace
  Write-Host ("[FAIL] {0}" -f $_.Exception.Message) -ForegroundColor Red
  Write-Host "[HINT] log file: ...  |  press Retry - successful steps are not repeated"
  exit 1
}
[Console]::OutputEncoding = UTF8      # safe unicode output for the panel
$cfg = [ordered]@{ Name='...'; DisplayName='...'; Domain='...'; BackendPort=8083;
                   SourceType='zip'; SourcePath='...'; DeployPath='...';
                   SslCertPath='...'; SslKeyPath='...'; NginxPath='...'; ... }
# Administrator check  → if not elevated: [FAIL] + exit 1
# Start-Transcript     → C:\ProgramData\DeployGuard\logs\<name>-<stage>.log
```

همهٔ اسکریپت‌ها با چهار برچسب رنگی استاندارد صحبت می‌کنند: سبز `[OK]`، زرد `[WARN]` (مانع ادامه نیست)، قرمز `[FAIL]` (توقف فوری با exit ۱) و `[RESULT]` جمع‌بندی. اسکریپت سلامت هم خطوط `[CHECK]` می‌دهد که پنل آن‌ها را به گزارش رنگی چندسطحی تبدیل می‌کند. هر [FAIL] با [HINT] همراه است: مسیر فایل ترنسکریپت + یادآوری دکمهٔ «تلاش دوباره»؛ و چون اسکریپت‌ها idempotent نوشته شده‌اند، در اجرای مجدد فقط مراحل باقی‌مانده اجرا می‌شوند، نه از صفر.

شش اسکریپت استقرار — به ترتیب اجرا

نام فایل و نام مرحله در پنل (ستون کناری هنگام اجرا) و کاری که هر اسکریپت دقیقاً انجام می‌دهد:

۱ `01-Preflight.ps1` — بررسی‌های پیش‌نیاز

هفت کنترل لحظهٔ اجرا: **۱) آزاد بودن پورت بک‌اند** — اگر اشغال باشد، صاحب پورت با نام پردازش، PID و مسیر فایل اجرایی شناسایی و با استعلام Win۳۲_Service مشخص می‌شود سرویس ویندوز است یا نه؛ اگر سرویسِ خودِ همین سامانه باشد (حالت استقرار مجدد/به‌روزرسانی) فقط هشدار زرد می‌دهد و ادامه می‌دهد، اما پردازش بیگانه یعنی [FAIL] همراه با نام پردازش و دستور دقیق توقف (`Stop-Service` یا `Stop-Process -Id`)؛ **۲)** وجود فایل گواهی SSL؛ **۳)** وجود کلید خصوصی؛ **٤)** وجود `nginx.exe` و نسخهٔ آن؛ **۵)** در دسترس بودن Python و Node.js در PATH با نمایش نسخه؛ **٦)** رزولوشن DNS دامنه و هشدار اگر IP نتیجه با IP کارت‌های شبکه سرور یکی نباشد؛ **۷)** وجود منبع کد (مسیر یا ZIP) و آماده بودن درایو مقصد — پوشهٔ والد مسیر استقرار در صورت نبود خودکار ساخته می‌شود. حتی یک [FAIL] یعنی توقف کل پلن پیش از هر تغییری روی سرور.

۲ `02-Prepare.ps1` — آماده‌سازی پوشه‌ها و کد

ابتدا ساختار پوشهٔ استاندارد را می‌سازد: `nginx`، `service`، `logs`، `www`، `frontend`، `backend` و `scripts`. سپس منبع کد را سر جای خود می‌برد: در حالت «مسیر» با `robocopy /MIR` از backend و frontend کپی می‌کند؛ در حالت «ZIP» ابتدا سالم بودن آرشیو را کنترل می‌کند (شمارش entryها)، پوشه‌های موقت قدیمی‌تر از ۶ ساعت را پاک می‌کند، با `tar.exe` (یا Expand-Archive) استخراج می‌کند، ریشهٔ پروژه را حتی درون پوشه‌های تودرتوی ZIP (تا ۵ سطح) پیدا می‌کند و بعد robocopy می‌زند. در پایان فایل `backend\.env` را با UTF-۸ بدون BOM می‌نویسد: APP_NAME، PORT، APP_DOMAIN و ENV=production. `C:\ProgramData\DeployGuard\tools` در مسیر استقرار + پوشهٔ ابزار.

۳ `03-Backend.ps1` — استقرار بک‌اند FastAPI

چهار کار: **۱)** ساخت محیط مجازی `.venv` با `python -m venv` (اگر از قبل نباشد)؛ **۲)** نصب وابستگی‌ها — ارتقای pip و اجرای `pip install -r requirements.txt`؛ نبودِ requirements.txt یعنی فقط uvicorn نصب می‌شود با هشدار؛ **۳)** آماده‌سازی سرویس ویندوز: اگر `WinSW-x64.exe` در پوشهٔ ابزار نباشد نسخهٔ ۲.۱۲.۰ دانلود می‌شود، به `service\<name>-service.exe` کپی و فایل تعریف سرویس `<name>-service.xml` نوشته می‌شود (اجرای uvicorn روی پورت بک‌اند، استارت خودکار، ری‌استارت خودکار پس از خطا با تأخیر ۵ و ۱۰ ثانیه)؛ **٤)** نصب و استارت سرویس — سرویس قبلی هم‌نام ابتدا توقف و حذف می‌شود — و سپس حداکثر ۹۰ ثانیه منتظر پاسخ پورت می‌ماند؛ نشدن = خطا با ارجاع به لاگ‌های wrapper.

٤ `04-Frontend.ps1` — ساخت فرانت‌اند React

اگر `frontend\package.json` وجود نداشته باشد با هشدار خارج می‌شود و سایت فعلی دست‌نخورده می‌ماند. وگرنه: `npm ci` (وقتی package-lock.json هست) یا `npm install`، سپس `npm run build`؛ پیدا نشدن خروجی `dist` یا `build` یعنی خطا. در پایان محتوای `www` را خالی و خروجی production build را با robocopy /MIR داخل آن منتشر می‌کند — همان پوشه‌ای که Nginx سرو می‌کند.

۵ `05-Nginx.ps1` — پیکربندی Nginx و SSL

شش کار به‌ترتیب: **۱)** گرفتن قفل سراسری `C:\ProgramData\DeployGuard\nginx.lock` تا دو استقرار هم‌زمان کانفیگ مشترک را خراب نکنند (قفل یتیمِ قدیمی‌تر از ۱۰ دقیقه خودکار حذف و حداکثر ۶۰ ثانیه برای آزادشدن منتظر می‌ماند)؛ **۲)** کنترل `nginx.conf` برای وجود `include conf.d/*.conf` و map وب‌سوکت و افزودن آن‌ها با بکاپ در صورت نبود؛ **۳)** اعتبارسنجی عمیق گواهی: خوانایی، انقضا (منقضی = خطا؛ کمتر از ۱۴ روز = هشدار)، تطابق CN/SAN با دامنه (نامتطابق = هشدار)، تک‌گواهی بودن بدون زنجیرهٔ میانی (هشدار) و تطابق modulus کلید و گواهی با openssl (نامتطابق = خطا)؛ **٤)** بکاپ کانفیگ قبلی دامنه با timestamp در `nginx-backups` و نوشتن `conf.d\<name>.conf`؛ **۵)** ساخت قواعد فایروال ۸۰ TCP و ۴۴۳ با نام DeployGuard HTTP؛ **٦)** تست با `nginx -t` — رد شدن یعنی بازگردانی خودکار کانفیگ قبلی و نمایش خطا با متن خطوط emerg — و در پایان reload اگر Nginx روشن است، وگرنه استارت آن.

۶ — `06-Verify.ps1` — اعتبارسنجی نهایی

بدون تغییر دادن چیزی فقط تأیید می‌کند: وضعیت سرویس `deployguard-<name>` باید Running باشد؛ پورت بک‌اند روی ۱۲۷.۰.۰.۱ باید پاسخ دهد (Test-NetConnection)؛ پروب HTTP بک‌اند روی `/docs` و در نبودش روی `/` زده می‌شود (شکست = هشدار، چون اپ ممکن است هنوز در حال بوت باشد)؛ و `https://دامنه` با curl.exe آزمایش می‌شود (۲۰۰/۳۰۱/۳۰۲ = سبز). در پایان جمع‌بندی استقرار چاپ می‌شود: آدرس سایت، آدرس API، نام سرویس، پوشه و فایل کانفیگ Nginx. هر مورد قرمز = exit ۱. همین اسکریپت پس از به‌روزرسانی هم (با نام U۶) اجرا می‌شود.

ترنسکریپت هر مرحله روی سرور — متن کامل خروجی هر اسکریپت (همان که در ترمینال پنل می‌بینید) در این فایل‌ها هم ذخیره می‌شود تا برای عیب‌یابی یا پیوست‌کردن به گزارش، همیشه در دسترس باشند:

مرحله	فایل ترنسکریپت در C:\ProgramData\DeployGuard\logs
۰۱ — بررسی‌های پیش‌نیاز	`<name>-preflight.log`
۰۲ — آماده‌سازی پوشه‌ها و کد	`<name>-prepare.log`
۰۳ — استقرار بک‌اند	`<name>-backend.log`
۰۴ — ساخت فرانت‌اند	`<name>-frontend.log`
۰۵ — پیکربندی Nginx	`<name>-nginx.log`
به‌روزرسانی (U۱ تا U۵)	`<name>-update-preflight / -prepare / -backend / -frontend / -switch.log`
بازگشت به نسخهٔ سالم	`<name>-rollback.log`
۰۶ / U۶ — اعتبارسنجی نهایی	ترنسکریپت ندارد — فقط کنترل می‌کند و چیزی تغییر نمی‌دهد

اسکریپت‌های مدیریتی و نگهداری

کنترل سرویس — Start / Stop / Restart

اسکریپت کوتاه کنترل سرویس (از تب «سرویس‌ها» اجرا می‌شود): سرویس `deployguard-<name>` را با Get-Service پیدا می‌کند و بسته به عملیات `Start-Service`، `Stop-Service` یا `Restart-Service -Force` اجرا می‌کند و وضعیت نهایی را با [OK] چاپ می‌کند؛ سرویس ناموجود = [FAIL] با پیام روشن.

۰۷-Health.ps۱ — سلامت چندسطحی

برای هر سطح یک خط `کلید | وضعیت | [CHECK] جزئیات` می‌دهد که پنل به گزارش رنگی تبدیل می‌کند: سرویس ویندوز؛ بک‌اند (زنجیرهٔ /docs ← /api/health ← health / با تایم‌اوت ۸ ثانیه)؛ دیتابیس (اول JSON پاسخ سلامت اپ، بعد DATABASE_URL در env:. sqlite = وجود فایل، postgres/mysql/mssql = تست TCP سه‌ثانیه‌ای)؛ Nginx (پردازش در حال اجرا + فایل کانفیگ دامنه)؛ HTTPS واقعی روی دامنه؛ و هشدارهای نگهداری: فضای دیسک (کمتر از ۱ گیگابایت = قرمز)، انقضای گواهی SSL و حجم لاگ‌ها (بیش از ۱ گیگابایت = قرمز). جمع‌بندی: تعداد ok / warn / fail.

چرخش لاگ‌ها

فایل‌های `.log` و `.txt` قدیمی‌تر از ۷ روز را در دو مقصد (لاگ‌های خود سامانه + لاگ‌های استقراربان در ProgramData) با `Compress-Archive` در `archive\logs-<تاریخ>.zip` آرشیو و اصل را حذف می‌کند؛ فایل‌های درگیرِ سرویس با هشدار رد می‌شوند؛ آرشیوهای قدیمی‌تر از ۳۰ روز پاک و حجم آزادشده به MB گزارش می‌شود.

حذف کامل (Uninstall)

سه گام: توقف و حذف سرویس WinSW؛ تغییرنام کانفیگ Nginx به `<name>.conf.removed_<timestamp>` و سپس reload + `nginx -t`؛ آرشیو لاگ‌ها به `C:\ProgramData\DeployGuard\backups\<name>_<timestamp>` و در پایان حذف کامل پوشهٔ سامانه با همهٔ محتویات.

اسکریپت‌های به‌روزرسانی نسخه‌دار (U۱ تا U۶) و بازگشت

ویزارد به‌روزرسانی هم یک پلن شش‌اسکریپتی می‌سازد با همان اسکلت مشترک؛ تفاوت کلیدی، **غیرمخرب بودن** آن است: نسخهٔ فعال تا مرحلهٔ سوییچ نهایی (U۵) دست‌نخورده در سرویس باقی می‌ماند.

اسکریپت	مرحلهٔ پنل	دقیقاً چه می‌کند
`U1-Update-Preflight.ps1`	بررسی پیش از به‌روزرسانی	وجود پوشهٔ استقرار و منبع جدید را کنترل می‌کند؛ نبود سرویس فعلی = هشدار؛ اگر پردازش بیگانه‌ای پورت بک‌اند را گرفته باشد هشدار با دستور توقف می‌دهد (سوییچ نهایی به این پورت نیاز دارد) و حداقل ۱ گیگابایت فضای آزاد درایو استقرار را شرط می‌داند.
`U2-Stage-Release.ps1`	آماده‌سازی نسخهٔ جدید	ساخت `releases\<tag>\` با backend/frontend/www و پاک‌سازی بقایای تلاش ناموفق قبلی با همان tag؛ استخراج/کپی منبع جدید (همان منطق مرحلهٔ ۰۲)؛ انتقال وضعیت اجرایی از نسخهٔ فعال: `.env`، فایل‌های دیتابیس sqlite و پوشه‌های media/uploads/static؛ ثبت `release.json`. نسخهٔ در حال اجرا هیچ تغییری نمی‌کند.
`U3-Backend-Deps.ps1`	وابستگی‌های بک‌اند	venv مشترک (خارج از releases) تا کش pip و وضعیت حفظ شود) را آماده و requirements.txt نسخهٔ جدید را در همان venv نصب می‌کند؛ سرویس همچنان نسخهٔ فعلی را اجرا می‌کند.
`U4-Frontend-Build.ps1`	ساخت فرانت‌اند	npm ci/install و npm run build داخل پوشهٔ نسخهٔ جدید؛ خروجی فقط در `releases\<tag>\www` مرحله‌بندی می‌شود و منتشر نمی‌شود؛ نبود package.json یعنی سایت فعلی حفظ می‌شود.
`U5-Safe-Switch.ps1`	سوییچ ایمن	تنها مرحلهٔ مخرب: بکاپ XML سرویس (`.bak_<tag>`) ← انتشار www جدید ← توقف سرویس ← تغییر `<workingdirectory>` به نسخهٔ جدید ← uninstall/install سرویس ← استارت؛ شکست در استارت یا نپاسخ‌گویی پورت تا ۹۰ ثانیه = بازگشت خودکار به نسخهٔ قبلی (بازگردانی XML + نصب + استارت)؛ در پایان ثبت tag = `releases\.active`.
`U6-Verify.ps1`	اعتبارسنجی نهایی	همان Verify-۰۶: سرویس، پورت، پروب HTTP و HTTPS دامنه.

اسکریپت «**بازگشت به نسخهٔ سالم**» (از تب «نسخه‌ها») هم الگوی مشابهی دارد: بکاپ XML سرویس، توقف سرویس، تغییر مقصد به `releases\<tag>\backend` نسخهٔ سالم، نصب و استارت با بازگردانی خودکار تعریف سرویس در صورت خطا، گیت ۹۰ ثانیه‌ای پورت، انتشار دوبارهٔ www همان نسخه و ثبت `.active`. یک هشدار مهم هم چاپ می‌کند: بازگشت فقط «کد» را برمی‌گرداند و تغییرات اسکیمای دیتابیس نسخهٔ جدیدتر خودکار برنمی‌گردند.

دو فایل کانفیگی که همراه اسکریپت‌ها تولید می‌شوند

conf.d\<name>.conf — کانفیگ Nginx دامنه

ریدایرکت دائمی ۸۰ به ۴۴۳؛ listener با ssl/http۲ و TLS۱.۲/۱.۳؛ هدرهای امنیتی (HSTS، nosniff، SAMEORIGIN)؛ client_max_body_size برابر UploadLimitMb ویزارد؛ gzip؛ سرو SPA با try_files؛ کش ۳۰ روزهٔ فایل‌های استاتیک؛ پروکسی /api/ به 127.0.0.1:<port> با پشتیبانی وب‌سوکت (Upgrade) و تایم‌اوت‌های ProxyTimeoutSec؛ مسدودسازی یا انتشار docs/ و openapi.json/ بسته به BlockDocs؛ و /healthz با پاسخ ۲۰۰ برای مانیتورینگ.

service\<name>-service.xml — تعریف سرویس WinSW

شناسهٔ deployguard-<name>؛ اجرای backend\.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port <port> با workingdirectory پوشهٔ بک‌اند؛ استارت Automatic؛ ری‌استارت خودکار پس از خطا با تأخیر ۵ و ۱۰ ثانیه و resetfailure یک‌ساعته؛ لاگ roll-by-size ده‌مگابایتی تا ۸ فایل در پوشهٔ logs؛ تایم‌اوت توقف ۱۵ ثانیه. در سوییچ به‌روزرسانی یا بازگشت، فقط مقدار workingdirectory همین فایل تغییر می‌کند — بقیهٔ تعریف ثابت می‌ماند.

پیوست فصل — متن کامل یک اسکریپت واقعی: 05-Nginx.ps1

در ادامه **متن کامل و بدون هیچ حذفی** یکی از شش اسکریپت استقرار را می‌بینید — همان 05-Nginx.ps1 که در ستون اجرا با نام مرحلهٔ «پیکربندی Nginx و SSL» اجرا می‌شود. این دقیقاً خروجی پنل برای سامانهٔ نمونهٔ university-scheduler روی دامنهٔ barnamehyar360.ir است: تمام مقادیر داخل بلوک $cfg همان چیزی است که در گام ۱ ویزارد وارد شده و تغییر هر مقدار در ویزارد، یعنی بازتولید همین متن با مقادیر جدید. پنج اسکریپت دیگر هم دقیقاً همین ساختار را دارند؛ فقط بلوک منطقی میانی هر کدام (بررسی‌ها، کپی کد، venv و سرویس، build فرانت‌اند، اعتبارسنجی) فرق می‌کند.

راهنمای خواندن: هر بخش از اسکریپت با خط نظر # ---- شروع می‌شود و ترتیب بخش‌ها همان ترتیب جدول‌های همین فصل است. برچسب‌های خروجی هم رنگ معنا دارند: سبز [OK]، زرد [WARN]، قرمز [FAIL] و جمع‌بندی [RESULT]. متن اسکریپت‌ها عمداً انگلیسی است تا کپی‌کردن خطای دقیق برای جستجو و پیوست‌کردن به گزارش ساده‌تر باشد؛ توضیح فارسی هر مرحله را در همین فصل و در تب «خروجی استقرار» پنل می‌خوانید.

05-Nginx.ps1 · part 1/4 — common skeleton: params, trap [FAIL]/[HINT], $cfg block, admin check, transcript

```
param([switch]$Silent)
$ErrorActionPreference = 'Stop'
$script:LogPath = $null
# clean fatal-error reporting: one red [FAIL] line instead of a raw PowerShell stack
trap {
  Write-Host ("[FAIL] {0}" -f $_.Exception.Message) -ForegroundColor Red
  Write-Host "[HINT] the full transcript log with every detail is available in the panel (step output) and in the log file shown above" -ForegroundColor Yellow
  if ($script:LogPath) { Write-Host ("[HINT] log file: {0}" -f $script:LogPath) -ForegroundColor Yellow }
  Write-Host "[HINT] fix the cause, then press the Retry button in the panel - successful steps are not repeated" -ForegroundColor Yellow
  try { Stop-Transcript | Out-Null } catch {}
  exit 1
}
# keep console output UTF-8 so the panel reads every line correctly
try { [Console]::OutputEncoding = New-Object System.Text.UTF8Encoding($false) } catch {}
$cfg = [ordered]@{
  Name           = 'university-scheduler'
  DisplayName    = 'university-scheduler'
  Domain         = 'barnamehyar360.ir'
  BackendPort    = 8083
  SourceType     = 'zip'
```

```
  SourcePath      = 'C:\university-scheduler\university-scheduler.zip'
  DeployPath      = 'C:\university-scheduler\university-scheduler'
  SslCertPath     = 'C:\SSL\barnamehyar360\certificate.pem'
  SslKeyPath      = 'C:\SSL\barnamehyar360\private-key.pem'
  NginxPath       = 'C:\nginx'
  UploadLimitMb   = 100
  ProxyTimeoutSec = 120
  BlockDocs       = $false
}

Write-Host ""
Write-Host "============================================================" -ForegroundColor Cyan
Write-Host "  DeployGuard :: Nginx :: Site Configuration & Reload" -ForegroundColor Cyan
Write-Host "  Target: $($cfg.DisplayName)  ($($cfg.Domain))" -ForegroundColor Gray
Write-Host "  Time  : $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')" -ForegroundColor Gray
Write-Host "============================================================" -ForegroundColor Cyan


# ---- Administrator privilege check -------------------------------------
$principal = New-Object
Security.Principal.WindowsPrincipal([Security.Principal.WindowsIdentity]::GetCurrent())
if (-not $principal.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
  Write-Host "[FAIL] This script must be run as Administrator." -ForegroundColor Red
  Write-Host "        Re-run the panel from an elevated PowerShell, or right-click → Run as
administrator."
  exit 1
}
Write-Host "[OK] Administrator privileges confirmed"

$logDir = 'C:\ProgramData\DeployGuard\logs'
if (-not (Test-Path $logDir)) { try { New-Item -ItemType Directory -Path $logDir -Force | Out-Null
} catch {} }
$script:LogPath = 'C:\ProgramData\DeployGuard\logs\university-scheduler-nginx.log'
try { Start-Transcript -Path 'C:\ProgramData\DeployGuard\logs\university-scheduler-nginx.log' -
Append | Out-Null } catch { Write-Host "[WARN] transcript unavailable: $($_.Exception.Message)" -
ForegroundColor Yellow }


function Invoke-Nginx([string[]]$nginxArgs) {
  $prevEap = $ErrorActionPreference; $ErrorActionPreference = 'Continue'
  $out = ((& $nginxExe @nginxArgs 2>&1) | Out-String).Trim()
  $code = $LASTEXITCODE
  $ErrorActionPreference = $prevEap
  return [pscustomobject]@{ Output = $out; Exit = $code }
}
```

05-Nginx.ps1 · part 2/4 — global nginx config lock + nginx.conf patch (include conf.d + websocket map)

```
$nginxExe  = Join-Path $cfg.NginxPath 'nginx.exe'
if (-not (Test-Path $nginxExe)) { throw "nginx.exe not found at $nginxExe - install Nginx or fix
the path in wizard step 1" }

$confDir   = Join-Path $cfg.NginxPath 'conf\conf.d'
$backupDir = Join-Path $cfg.DeployPath 'nginx-backups'
if (-not (Test-Path $confDir))   { New-Item -ItemType Directory -Path $confDir   -Force | Out-Null
}
if (-not (Test-Path $backupDir)) { New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
}

# ---- cross-deployment lock: two deployments must never rewrite shared nginx state at once ----
$lockFile = 'C:\ProgramData\DeployGuard\nginx.lock'
```

```
New-Item -ItemType Directory -Path (Split-Path $lockFile) -Force | Out-Null
# a lock left over from a crashed run expires after 10 minutes
if ((Test-Path $lockFile) -and ((Get-Item $lockFile -ErrorAction SilentlyContinue).LastWriteTime -
lt (Get-Date).AddMinutes(-10))) {
  Remove-Item $lockFile -Force -ErrorAction SilentlyContinue
  Write-Host "[WARN] stale nginx lock (older than 10 min) removed" -ForegroundColor Yellow
}
$lockDeadline = (Get-Date).AddSeconds(60)
while ((Test-Path $lockFile) -and (Get-Date) -lt $lockDeadline) {
  Write-Host "[..]   another deployment is changing the shared Nginx config - waiting for the lock"
-ForegroundColor Yellow
  Start-Sleep 3
}
if (Test-Path $lockFile) { throw "the shared Nginx config is locked by another deployment (lock
file $lockFile) - wait a minute and retry" }
New-Item -ItemType File -Path $lockFile -Force | Out-Null
Write-Host "[OK]   nginx config lock acquired"

# ensure 'include conf.d/*.conf;' + websocket map inside http{} of nginx.conf ------
$mainConf  = Join-Path $cfg.NginxPath 'conf\nginx.conf'
if (-not (Test-Path $mainConf)) { throw "nginx.conf not found at $mainConf - check the Nginx path
in wizard step 1" }
$content   = Get-Content $mainConf -Raw
$includeLine   = 'include conf.d/*.conf;'
$mapLine       = 'map $http_upgrade $connection_upgrade'
$needPatch     = $false
$patchLines    = @()
if ($content -notmatch [regex]::Escape($includeLine)) { $needPatch = $true; $patchLines += '    ' +
$includeLine }
if ($content -notmatch [regex]::Escape($mapLine)) {
  $needPatch = $true
  $patchLines += '    map $http_upgrade $connection_upgrade {', "        default upgrade;", "
''      close;", '    }'
}
if ($needPatch) {
  Copy-Item $mainConf "$mainConf.bak_deployguard" -Force
  $insert = ($patchLines -join [Environment]::NewLine)
  $patched = $content -replace '(?m)^(\s*http\s*\{)', ('$1' + [Environment]::NewLine + $insert)
  Set-Content -Path $mainConf -Value $patched -Encoding ASCII
  Write-Host "[OK]   conf.d include + websocket map added to nginx.conf (backup saved)"
} else {
  Write-Host "[OK]   nginx.conf already includes conf.d and the websocket map"
}
```

05-Nginx.ps1 · part 3/4 — deep SSL certificate validation (expiry, domain, key modulus, chain)

```
# ---- SSL certificate validation (expiry, domain, key match, chain) ---------------
if (-not (Test-Path $cfg.SslCertPath)) { throw "SSL certificate not found: $($cfg.SslCertPath) -
copy the .pem certificate or fix the path in wizard step 1" }
if (-not (Test-Path $cfg.SslKeyPath))  { throw "SSL private key not found: $($cfg.SslKeyPath) -
copy the .key file or fix the path in wizard step 1" }
try {
  $cert = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
$cfg.SslCertPath
} catch { throw "SSL certificate could not be read: $($_.Exception.Message) - the file may be
corrupt or in an unsupported format" }
$daysLeft = ($cert.NotAfter - (Get-Date)).Days
if ($daysLeft -lt 0) { throw ("SSL certificate EXPIRED on {0} - renew the certificate, update
wizard step 1, then retry" -f $cert.NotAfter.ToString('yyyy-MM-dd')) }
if ($daysLeft -le 14) {
  Write-Host ("[WARN] SSL certificate expires in {0} days ({1}) - renew it soon to avoid an outage"
```

```
-f $daysLeft, $cert.NotAfter.ToString('yyyy-MM-dd')) -ForegroundColor Yellow
} else {
  Write-Host ("[OK]   SSL certificate valid until {0} ({1} days left)" -f
$cert.NotAfter.ToString('yyyy-MM-dd'), $daysLeft)
}
$cn = ''
if ($cert.Subject -match 'CN\s*=\s*([^,]+)') { $cn = $Matches[1].Trim() }
$sanExt = ($cert.Extensions | Where-Object { $_.Oid.FriendlyName -eq 'Subject Alternative Name' } |
Select-Object -First 1)
$san = if ($sanExt) { ($sanExt.Format($false) -replace '^[^=]+=', '') } else { '' }
if ($cn -ne $cfg.Domain -and ($san -notmatch [regex]::Escape($cfg.Domain))) {
  Write-Host ("[WARN] certificate subject '{0}' may not cover domain '{1}' (SAN: {2}) - browsers
may warn" -f $cn, $cfg.Domain, ($(if ($san) { $san } else { 'none' }))) -ForegroundColor Yellow
} else {
  Write-Host "[OK]   certificate matches the domain"
}
$rawPem = Get-Content $cfg.SslCertPath -Raw
$certCount = ([regex]::Matches($rawPem, '-----BEGIN CERTIFICATE-----')).Count
if ($certCount -le 1) { Write-Host "[WARN] certificate file contains a single certificate - the
intermediate chain may be missing (browsers ok, some clients fail)" -ForegroundColor Yellow }
$openssl = Get-Command openssl.exe -ErrorAction SilentlyContinue
if ($openssl) {
  $prevEap = $ErrorActionPreference; $ErrorActionPreference = 'Continue'
  $cMod = ((& openssl x509 -noout -modulus -in $cfg.SslCertPath 2>$null) | Out-String).Trim()
  $kMod = ((& openssl rsa  -noout -modulus -in $cfg.SslKeyPath  2>$null) | Out-String).Trim()
  $ErrorActionPreference = $prevEap
  if ($cMod -and $kMod) {
    if ($cMod -ne $kMod) { throw "the private key does not match the certificate - you probably
mixed up files of another domain; check wizard step 1" }
    Write-Host "[OK]   private key matches the certificate (openssl modulus)"
  } else { Write-Host "[WARN] key/cert pair check skipped (openssl could not read the files)" -
ForegroundColor Yellow }
}
```

05-Nginx.ps1 · part 4/4 — write per-domain site conf, firewall rules, nginx -t & reload

```
# ---- write the site configuration (per-domain file) -----------------------------
$confContent = @'
# ============================================================
# MahDeploy :: university-scheduler
# domain  : barnamehyar360.ir
# backend : http://127.0.0.1:8083  (FastAPI / uvicorn)
# frontend: static build served below
# managed by MahDeploy - do not edit manually
# ============================================================
server {
    listen 80;
    server_name barnamehyar360.ir;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name barnamehyar360.ir;

    ssl_certificate      C:\SSL\barnamehyar360\certificate.pem;
    ssl_certificate_key  C:\SSL\barnamehyar360\private-key.pem;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers          ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-
AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;
    ssl_prefer_server_ciphers on;
    ssl_session_cache    shared:university-scheduler_ssl:10m;
```

```
    ssl_session_timeout  10m;

    # ---------- security headers ----------
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options    "nosniff" always;
    add_header X-Frame-Options           "SAMEORIGIN" always;
    add_header Referrer-Policy           "strict-origin-when-cross-origin" always;

    client_max_body_size 100m;
    gzip on;
    gzip_types text/plain text/css application/json application/javascript image/svg+xml;

    # ---------- frontend (React static build, SPA fallback) ----------
    root  C:\university-scheduler\university-scheduler\www;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # ---------- backend (FastAPI) ----------
    location /api/ {
        proxy_pass http://127.0.0.1:8083;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        # websocket / long-polling friendly
        proxy_set_header Upgrade           $http_upgrade;
        proxy_set_header Connection        $connection_upgrade;
        proxy_read_timeout  120s;
        proxy_send_timeout  120s;
        proxy_connect_timeout 30s;
        proxy_buffering off;
    }

    # ---------- API docs (public) ----------
    location /docs {
        proxy_pass http://127.0.0.1:8083;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /openapi.json {
        proxy_pass http://127.0.0.1:8083;
        proxy_set_header Host $host;
    }

    # ---------- health endpoint for the panel ----------
    location = /healthz {
        access_log off;
        return 200 "healthy";
    }
}

'@
$confPath = Join-Path $confDir ('{0}.conf' -f $cfg.Name)
$prevConf = $null
if (Test-Path $confPath) {
```

```
  $ts = Get-Date -Format 'yyyyMMdd-HHmmss'
  $prevConf = Join-Path $backupDir ("{0}.conf.{1}" -f $cfg.Name, $ts)
  Copy-Item $confPath $prevConf -Force
  Write-Host "[OK]   previous config backed up: $prevConf"
}
Set-Content -Path $confPath -Value $confContent -Encoding ASCII
Write-Host "[OK]   site config written: $confPath"

# firewall ------------------------------------------------------------------
foreach ($port in 80,443) {
  $rule = Get-NetFirewallRule -DisplayName "DeployGuard HTTP $port" -ErrorAction SilentlyContinue
  if (-not $rule) {
    New-NetFirewallRule -DisplayName "DeployGuard HTTP $port" -Direction Inbound -Protocol TCP -
LocalPort $port -Action Allow | Out-Null
    Write-Host "[OK]   firewall rule added for TCP $port"
  }
}

# test & reload -------------------------------------------------------------
$test = Invoke-Nginx @('-t')
Write-Host $test.Output
if ($test.Exit -ne 0) {
  if ($prevConf) {
    Copy-Item $prevConf $confPath -Force
    Write-Host "[WARN] new config was rejected - the previous config was restored automatically" -
ForegroundColor Yellow
    Invoke-Nginx @('-t') | Out-Null
  }
  $emergLines = @($test.Output -split "\r?\n" | Where-Object { $_ -match 'emerg|crit|\[error\]' } |
Select-Object -First 3)
  $detail = if ($emergLines) { $emergLines -join ' | ' } else { $test.Output }
  throw "nginx configuration test failed (exit $($test.Exit)): $detail - fix the reported line and
retry (a backup of the previous config is in $backupDir)"
}
Write-Host "[OK]   nginx configuration test passed"

$running = Get-Process nginx -ErrorAction SilentlyContinue
if ($running) {
  $reload = Invoke-Nginx @('-s','reload')
  if ($reload.Exit -ne 0) { throw "nginx reload failed (exit $($reload.Exit)): $($reload.Output)" }
  Write-Host "[OK]   nginx reloaded"
} else {
  Start-Process -FilePath $nginxExe -WorkingDirectory $cfg.NginxPath -WindowStyle Hidden
  Start-Sleep 2
  Write-Host "[OK]   nginx started"
}

Remove-Item $lockFile -Force -ErrorAction SilentlyContinue
Write-Host "[RESULT] Nginx site is live at https://$($cfg.Domain)" -ForegroundColor Green
try { Stop-Transcript | Out-Null } catch {}
```

همین متن را در سه جا می‌توانید ببینید و کپی کنید: در گام «تولید پلن» پیش از اجرا (قابل مرور، کپی و دانلود)، در تب «اسکریپت‌های PowerShell» پنجرهٔ جزئیات سامانه، و روی سرور در فایل `scripts\05-Nginx.ps1` کنار بقیهٔ اسکریپت‌های پلن. پنل هر اسکریپت را با `powershell.exe -NoProfile -NonInteractive -ExecutionPolicy Bypass` اجرا می‌کند و خروجی خط‌به‌خط به همان مرحلهٔ پلن برمی‌گردد؛ هیچ‌کدام از این دستورها به‌صورت دستی و جدا از پنل اجرا نمی‌شوند.

CHAPTER 06

مدیریت سامانه‌های مستقرشده

چرخه حیات پس از استقرار: سلامت چندسطحی، به‌روزرسانی نسخه‌دار و بازگشت، کنترل سرویس‌ها، جزئیات فنی و حذف کامل.

تب «سامانه‌ها» برای هر استقرار یک کارت می‌سازد: نام نمایشی و دامنه، پورت بک‌اند، مسیر پوشه استقرار، زمان آخرین استقرار و نشان وضعیت. چهار دکمه پایین هر کارت، کل چرخه حیات را پوشش می‌دهند و دکمه «راهنمای مدیریت سامانه‌ها» در بالای همین صفحه، معادل داخل‌پنلیِ این فصل است. اگر فقط به خاموش‌وروشن‌کردن سرویس‌ها نیاز دارید، تب «سرویس‌ها» همان کار را با ظاهری ساده‌تر انجام می‌دهد.

نشان وضعیت	معنا	واکنش پیشنهادی
فعال	سرویس ویندوز در حال اجراست و سایت پاسخ می‌دهد	هیچ کاری لازم نیست؛ از لینک دامنه استفاده کنید
متوقف	سرویس با دکمه توقف، Stop شده است	در صورت نیاز، با دکمه Play شروع کنید
ناموفق	استقرار یا اجرای سرویس با خطا مواجه شده	جزئیات ← تب «خروجی استقرار» ← علت را از لاگ قرمز بخوانید
در حال استقرار	عملیات استقرار در جریان است	صبر کنید تا مراحل کامل شوند

بررسی سلامت چندسطحی — باز بودن پورت یعنی سالم؟

باز بودن پورت فقط نشان می‌دهد چیزی روی آن پورت گوش می‌دهد؛ نه اینکه سامانه واقعاً کار کند. برای همین دکمهٔ «**بررسی سلامت چندسطحی**» روی کارت هر سامانه، زنجیرهٔ کامل را از داخل به بیرون تست می‌کند و علاوه بر آن، چند هشدار نگهداری هم می‌دهد. نتیجه به‌صورت فهرست رنگی (سبز/زرد/قرمز) با توضیح هر سطح نمایش داده می‌شود:

سطح بررسی	دقیقاً چه چیزی تست می‌شود
سرویس ویندوز	سرویس `deployguard-<name>` در حال اجراست؟ (نه فقط پورت — خود سرویس)
بک‌اند	مسیر `/health` بک‌اند روی ۱۲۷.۰.۰.۱ پاسخ ۲۰۰ HTTP می‌دهد؟
دیتابیس	در پاسخ سلامت اپ، اتصال دیتابیس «connected» گزارش شده؟ (وابستگی‌های ضروری)
Nginx و کانفیگ دامنه	سرویس Nginx فعال است و فایل کانفیگ اختصاصی همین دامنه وجود دارد؟
HTTPS از مسیر دامنه	درخواست واقعی از `https://دامنه` پاسخ می‌دهد؟ (کل زنجیرهٔ SSL + Nginx + بک‌اند)
هشدارها	فضای دیسک، تاریخ انقضای گواهی SSL (کمتر از ۳۰ روز = زرد) و حجم لاگ‌ها (بیش از ۵۰۰ مگابایت = زرد)

چرخش لاگ‌ها: دکمهٔ «چرخش لاگ‌ها» در همان پنجرهٔ سلامت، لاگ‌های بزرگ هر سامانه را به آرشیو تاریخ‌دار منتقل می‌کند تا دیسک پر نشود؛ برای خودکاری‌سازی هم می‌توانید در ویندوز Task Schedule بسازید که اسکریپت تولیدشدهٔ «چرخش لاگ» را هفتگی اجرا کند.

به‌روزرسانی نسخه‌دار و بازگشت به نسخهٔ سالم

دکمهٔ «**به‌روزرسانی**» روی کارت هر سامانه، ویزاردی شش‌مرحله‌ای (مشابه استقرار اولیه) باز می‌کند با یک تفاوت کلیدی: نسخهٔ جدید در پوشهٔ جدا `releases\<tag>` آماده می‌شود و **نسخهٔ فعال تا مرحلهٔ سوییچ نهایی دست‌نخورده و در سرویس باقی می‌ماند** — اگر در آماده‌سازی یا تست نسخه جدید به مشکل بخورید، سایت کاربران هیچ وقفه‌ای نمی‌بیند.

versioned layout — C:\deployments\<name>

```
C:\deployments\docs-manager\
├── releases\           ← هر به‌روزرسانی یک پوشهٔ تاریخ‌دار
│   ├── 20260102-093000\   ← نسخه فعال (backend + www)
│   └── 20260210-141500\   ← نسخه‌های قبلی — برای بازگشت حفظ می‌شوند
├── logs\               ← لاگ‌ها مشترک و پیوسته
└── scripts\            ← اسکریپت‌های آخرین پلن
```

تاریخچه نسخه‌ها

در پنجره جزئیات، تب «نسخه‌ها» فهرست همه نسخه‌ها را با تاریخ، وضعیت (فعال/قبلی/معلق) و موجودبودن روی دیسک نشان می‌دهد؛ خروجی کامل هر استقرار هم در همان پنجره قابل مرور است.

بازگشت سریع (Rollback)

اگر نسخه جدید مشکل داشت، از تب «نسخه‌ها» دکمهٔ «بازگشت به این نسخه» را بزنید: سوییچ کانفیگ به نسخهٔ سالم + ری‌استارت سرویس، در چند ثانیه — بدون استقرار مجدد از ZIP.

اجرای ایمن و مجدد

هر اسکریپت «ایده‌پوتنت» نوشته شده: اجرای مجدد مرحلهٔ ناموفق، چیزی را خراب یا تکراری نمی‌کند. تغییر کانفیگ مشترک Nginx هم با قفل انجام می‌شود تا دو استقرار هم‌زمان کانفیگ هم را خراب نکنند.

محدودهٔ بازگشت

بازگشت، «کد» را برمی‌گرداند؛ تغییرات اسکیمای دیتابیس خودکار برنمی‌گردد — برای دیتابیس، مهاجرت جداگانه بگیرید (بکاپ قبل از به‌روزرسانی) و کانفیگ‌های محیطی را در نسخهٔ بازگشتی بررسی کنید.

تب «سرویس‌ها» ساده‌ترین جای پنل برای مدیریت روزمره است: فهرست همهٔ سامانه‌های مستقرشده با پورت بک‌اند و وضعیت زندهٔ هر کدام، همراه سه دکمه Start، Stop و Restart در همان ردیف. نشان سبز «روی پورت پاسخ می‌دهد» یعنی همین لحظه پردازشی روی آن پورت در حال شنیدن است؛ نشان قرمز «پورت بی‌پاسخ» یعنی سرویس ظاهراً بالا نیست. وضعیت پورت‌ها هر ۵ ثانیه به‌طور خودکار تازه می‌شود و دکمه بازآوری برای بررسی فوری هست. فقط سرویس‌های سامانه‌های مستقرشده با استقراربان از این‌جا کنترل می‌شوند و سرویس‌های سیستمی ویندوز دست‌نخورده می‌مانند؛ دکمه لینک، سامانه را در مرورگر باز می‌کند و دکمه چشم به تب «سامانه‌ها» برای کارهای تکمیلی می‌رود.

شروع / توقف / ری‌استارت

سه دکمه کنار کارت، سرویس ویندوز `deployguard-<name>` را مدیریت می‌کنند و نتیجه با اعلان (Toast) اعلام می‌شود. ری‌استارت برای اعمال تغییرات کد بک‌اند کافی است.

پنجره جزئیات

پنج تب دارد: «خروجی استقرار» (نتیجه ۶ اسکریپت با لاگ هرکدام)، «نسخه‌ها» (تاریخچه + بازگشت)، «اسکریپت‌های PowerShell» (کد کامل + کپی/دانلود)، «کانفیگ Nginx» (محتوای فایل conf اختصاصی دامنه) و «رویدادها» (لاگ‌های ثبت‌شده).

حذف کامل: با تأیید دومرحله‌ای انجام می‌شود — سرویس ویندوز حذف، کانفیگ Nginx پاک و ریلود می‌شود، لاگ‌ها آرشیو و پوشه سامانه (شامل دیتابیس خودِ اپ) حذف می‌گردد. این عملیات برگشت‌پذیر نیست؛ پیش از آن از پوشه `logs` و داده‌ها پشتیبان بگیرید.

CHAPTER 07

۰۷ امنیت و نگهداری

چند عادت ساده که پنل و سرور شما را سالم و امن نگه می‌دارد.

۱ رمز عبور

رمز پیش‌فرض `admin1234` را بلافاصله تغییر دهید و رمز قوی (حداقل ۸ کاراکتر) انتخاب کنید. نشست ورود ۱۲ ساعت اعتبار دارد و با امضای HMAC در کوکی محافظت می‌شود.

۲ دسترسی به پنل

پنل را فقط در شبکه مورد اعتماد در دسترس نگه دارید؛ برای انتشار از طریق اینترنت، جلوی پنل هم Nginx + SSL بگذارید و پورت ۳۰۰۰ را مستقیم باز نکنید.

۳ شفافیت اسکریپت‌ها

اسکریپت‌های PowerShell پیش از اجرا نمایش داده می‌شوند و اجرای استقرار نیازمند تأیید صریح شماست. اسکریپت‌ها به‌طور موقت در `%TEMP%\deployguard` نوشته و پس از اجرا حذف می‌شوند.

٤ پشتیبان‌گیری

دیتابیس پنل یک فایل است: `data\mahdeploy.db` — کپی همین فایل (یا کل پوشه پکیج) یعنی پشتیبان کامل از کاربران، استقرارها و لاگ‌ها. برای سامانه‌های مستقرشده هم پوشه هر اپ را جداگانه پشتیبان بگیرید.

۵ به‌روزرسانی و انتقال

برای انتقال به سرور جدید، کافی است کل پوشه پکیج را کپی کنید — همه‌چیز (حتی نشست‌ها و تنظیمات) حفظ می‌شود. نسخه‌های جدید پکیج را از صفحه Releases مخزن گیت‌هاب دریافت کنید.

٦ چرخش لاگ‌ها و پایش دوره‌ای

لاگ‌های هر سامانه در `logs` همان پوشه جمع می‌شوند؛ با دکمه «چرخش لاگ‌ها» (پنجره سلامت چندسطحی) لاگ‌های بزرگ به آرشیو تاریخ‌دار منتقل و فضا آزاد می‌شود. هفته‌ای یک‌بار «بررسی سلامت چندسطحی» را بزنید تا فضای دیسک، انقضای گواهی SSL (هشدار زیر ۳۰ روز) و حجم لاگ‌ها زیر نظر باشد.

۷ به‌روزرسانی امن اپ‌های مستقرشده

برای ارتقای هر سامانه از دکمه «به‌روزرسانی» استفاده کنید — نه استقرار مجدد: نسخه فعال می‌ماند، نسخه جدید در releases\<tag> آماده و تست می‌شود و در صورت مشکل، «بازگشت به نسخه» در چند ثانیه کار می‌کند. قبل از هر به‌روزرسانی، از پوشه اپ و دیتابیس آن پشتیبان بگیرید؛ تغییرات اسکیمای دیتابیس با بازگشت خودکار برنمی‌گردد.

فایروال: پنل هنگام استقرار، قاعده فایروال پورت بک‌اند را خودکار می‌سازد؛ برای خود پنل (پورت ۳۰۰۰) در صورت نیاز دستی اقدام کنید — دستور در فصل ۲ آمده است.

CHAPTER 08

عیب‌یابی و پرسش‌های رایج

۰۸

حالت‌های پرتکرار با علت و راه‌حل + راهنمای خواندن گزارش خطای پنل. برای ردیابی دقیق، لاگ‌ها بهترین دوست شما هستند: در پنل مسیر «سامانه‌ها ← جزئیات ← رویدادها» و روی دیسک پوشه logs پکیج و هر سامانه.

مشکل	علت احتمالی	راه‌حل
پنل بالا نمی‌آید / پیام «پورت اشغال»	پورت ۳۰۰۰ توسط برنامه دیگری استفاده می‌شود	با پورت دیگر اجرا کنید: `Start-MahDeploy.bat 3100`
بررسی دامنه قرمز است	رکورد A تنظیم نیست یا هنوز منتشر نشده	در پنل دامنه، رکورد A را روی IP سرور ثبت و پس از انتشار «بررسی مجدد» بگیرید
بررسی پورت قرمز است	پورت انتخابی اشغال است	از دکمه پیشنهاد پورت آزاد استفاده کنید یا عدد دیگری بدهید
اعتبارسنجی SSL شکست می‌خورد	مسیر فایل‌ها اشتباه است یا گواهی به دامنه تعلق ندارد	مسیر دقیق `.crt` و `.key` را کنترل و تطابق دامنه گواهی را بررسی کنید
استقرار وسط کار متوقف شد	شکست یکی از ۶ اسکریپت	کارت «تحلیل خطا» را بخوانید: متن خطا + علت + راه‌حل پیشنهادی؛ پس از رفع، «تلاش دوباره از مرحلهٔ ناموفق» را بزنید — مراحل موفق تکرار نمی‌شوند
به‌روزرسانی به مشکل خورد	خطا در آماده‌سازی یا سوییچ نسخه جدید	نسخه فعال دست‌نخورده باقی مانده؛ خروجی مرحله را بخوانید و دوباره تلاش کنید — یا از تب «نسخه‌ها» به نسخهٔ سالم برگردید
پیام «کانفیگ Nginx قفل است»	استقرار هم‌زمان دیگری در جریان است	یک دقیقه صبر کنید و دوباره تلاش کنید؛ قفل یتیم پس از ۱۰ دقیقه خودکار آزاد می‌شود
سلامت‌سنجی سطح دیتابیس زرد/قرمز است	اتصال دیتابیس اپ قطع یا کند است	لاگ بک‌اند و وضعیت سرویس دیتابیس را بررسی کنید؛ /health اپ باید database را connected گزارش کند
هشدار «گواهی SSL نزدیک انقضا»	گواهی کمتر از ۳۰ روز اعتبار دارد	گواهی جدید را در همان مسیر جایگزین و از تب «کانفیگ Nginx» یا ویزارد، کانفیگ را مجدد اعمال کنید
سایت باز نمی‌شود ولی سرویس فعال است	فایروال یا کانفیگ Nginx	قاعده فایروال پورت ۴۴۳ و ۸۰ و فایل `conf.d\<name>.conf` را بررسی کنید
رمز عبور را فراموش کردم	—	فایل `data\mahdeploy.db` را حذف کنید؛ پنل با رمز پیش‌فرض از نو راه‌اندازی می‌شود (تمام داده‌های پنل پاک می‌شود)

CHECKLIST

پیش از هر استقرار، این ٤ مورد را چک کنید

سی ثانیه بازبینی، از یک ساعت عیب‌یابی بهتر است. پس از استقرار هم نشان سبز پورت را در تب «سرویس‌ها» و نتیجه ۶ اسکریپت را در «سامانه‌ها ← جزئیات» مرور کنید.

۱ — سلامت سرور

در داشبورد، CPU و رم و دیسک سبز باشند، Nginx فعال باشد و فضای کافی دیسک موجود باشد.

۲ — دامنه و پورت

رکورد A دامنه به IP همین سرور اشاره می‌کند و پورت بک‌اند آزاد است — هر دو بررسی مرحله ۲ ویزارد سبز باشند.

۳ — گواهی SSL

فایل‌های گواهی و کلید در سرور کپی شده‌اند و «اعتبارسنجی گواهی SSL» بدون خطا تأیید می‌شود.

٤ — دسترسی Administrator

پنل با دسترسی مدیر اجرا شده باشد تا اسکریپت‌ها، سرویس ویندوز و فایروال واقعاً اعمال شوند؛ وگرنه همه‌چیز فقط شبیه‌سازی می‌شود.

استقراربان

MAHDEPLOY V1.3.1

از پیش‌بررسی تا ترمینال زنده، از گزارش دقیق خطا تا سلامت چندسطحی و بازگشت به نسخهٔ سالم — استقرار استاندارد FastAPI + React روی ویندوز سرور، بدون نصب هیچ پیش‌نیازی، با رابط فارسی و گزارش شفاف PowerShell.

۱ · داشبورد سرور
۲ · ویزارد استقرار و به‌روزرسانی
۳ · سلامت چندسطحی
۴ · نسخه‌بندی و بازگشت

سند رسمی راهنمای کاربر — نسخه ۱.۳.۱ — آبان ۱۴۰۵ — فونت سند: B Mitra

تهیه شده توسط شرکت ماه افزارهوشمند رسا — MahAI.ir

github.com/hjalilvand2004-coder/MahDeploy